

Revisiting ML Models for Intrusion Detection: Do Tree-Based Methods Win on Both Known and Zero-Day Attacks?

Indian Institute of Information Technology, Nagpur
Nagpur, India

1st Dhruv Singh 2nd Raghav Singh 3rd Devguru Tiwari 4th Nischal Yeremseti

Dept. of CSE

Dept. of DSA

Dept. of DSA

Dept. of AIML

dsingh19072005@gmail.com

byjusynr222@email.com

devgurutiwari@email.com

nischalyerm@email.com

Abstract—Recent benchmarks on intrusion detection systems have favored deep learning. Xu and Liu (2025) reported that MLP achieves the highest F1-score on known attacks and OCSVM performs best against zero-day threats on the CICIDS2017 dataset, omitting gradient-boosted tree ensembles entirely. We address this gap by evaluating four supervised tree models (Random Forest, LightGBM, XGBoost, CatBoost) and two unsupervised tree anomaly detectors (Isolation Forest, Extended Isolation Forest), all tuned with Optuna on held-out validation splits. CatBoost achieves a new state-of-the-art F1 of 0.9643 on known attacks, surpassing MLP (0.9446) by 2.0 points. Extended Isolation Forest reaches F1 of 0.7821 on zero-day attacks, surpassing OCSVM (0.7575) by 2.5 points. SHAP analysis reveals a fundamental paradigm difference: supervised tree models rely on port numbers and TCP window sizes (signature-like features that encode known attack identities), while unsupervised tree detectors rely on backward packet length and flow inter-arrival time (physical traffic anomalies that generalize across unseen attack types). Together, these results confirm a “double victory” for tree-based methods and provide a principled explanation for why deep learning approaches collapse on zero-day traffic.

Index Terms—Intrusion Detection System (IDS), Anomaly Detection, Gradient Boosting, CICIDS2017, Zero-Day Attacks, Isolation Forest, SHAP, CatBoost, XGBoost, LightGBM

I. INTRODUCTION

Network-based attacks have grown steadily since the widespread adoption of cloud infrastructure and IoT devices. Intrusion detection systems (IDS) form the primary automated layer of defense against such threats, and their detection accuracy carries direct operational consequences. Which class of models performs best for IDS is therefore a question with practical, not merely academic, weight.

Two broad IDS paradigms exist. Signature-based systems (S-IDS) match observed traffic against a catalog of known attack patterns and cannot detect novel threats by construction. Anomaly-based systems (A-IDS) instead model normal traffic and flag deviations, which in principle allows detection of attacks absent from training data. Within A-IDS, supervised models learn from labeled attack samples while unsupervised models train exclusively on benign traffic.

A recent benchmark by Xu and Liu [1] evaluated four models on CICIDS2017 [2]: MLP, a 1D CNN, One-Class

SVM (OCSVM), and Local Outlier Factor (LOF). They found that MLP ($F1 = 0.9446$) leads on known attacks while OCSVM ($F1 = 0.7575$) leads on zero-day generalization, and concluded that deep learning should be preferred for known-attack environments while OCSVM handles novel threats. That study excluded gradient-boosted trees and random forests entirely.

This exclusion is consequential. Grinsztajn et al. [3] benchmarked 45 tabular datasets and showed that tree ensembles consistently match or beat deep learning because tabular features are not invariant to rotation and trees naturally preserve the irregular, non-smooth decision boundaries that characterize such data. Network flow records in CICIDS2017 are exactly this kind of heterogeneous tabular data: byte counts, flag ratios, packet inter-arrival times, and duration fields operating on very different scales with no natural embedding structure.

We test a “double victory” hypothesis: supervised tree ensembles will outperform deep learning on known attacks, and unsupervised tree anomaly detectors will outperform OCSVM on zero-day attacks. Our contributions are:

- A controlled comparison of CatBoost, XGBoost, LightGBM, and Random Forest against the MLP and CNN baselines from Xu and Liu on CICIDS2017, using identical train/test splits.
- A comparison of Isolation Forest and Extended Isolation Forest against OCSVM and LOF on the same three held-out zero-day attack types (DoS slowloris, DoS Slowhttptest, Bot).
- Hyperparameter optimization via Optuna [4], absent in Xu and Liu, applied uniformly across all new models.
- SHAP-based feature attribution analysis across both supervised and unsupervised tree models, revealing a fundamental difference in how each paradigm encodes its detection logic.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the methodology. Section IV presents results. Section V discusses implications and concludes. Appendix A lists all final hyperparameter configurations.

II. LITERATURE REVIEW

CICIDS2017. Sharafaldin et al. [2] constructed CICIDS2017 to address the lack of realistic, labeled network traffic datasets. It captures five days of traffic under controlled conditions, spanning 14 attack categories and one benign class, with over 80 engineered flow-level features extracted by CICFlowMeter. It has since become a standard IDS benchmark. Mashaly et al. [13] evaluated KNN variants and LOF on CICIDS2017 with PCA-based feature reduction, showing that semi-supervised approaches trained on normal data alone can detect anomalies when feature spaces are well-chosen. Hizal et al. [14] applied deep learning tuned for cloud IDS scenarios on CICIDS2017 and reported strong aggregate accuracy, though without a zero-day holdout protocol.

Deep learning for IDS. Wang and Yang [15] proposed a CNN-LSTM architecture with reinforcement learning components, reaching 97.3% detection accuracy on real-time cloud traffic. Kale et al. [17] introduced a hybrid spatial-temporal deep learning IDS framework validated on multiple benchmark datasets. Tossou et al. [18] applied deep anomaly detection to IDS and reported improved generalization compared to classical methods. Wang et al. [16] demonstrated that purely supervised deep models are vulnerable to class imbalance, with overfitting risks that grow as attack class representation shrinks.

Tree-based methods on tabular data. Grinsztajn et al. [3] is the theoretical foundation for our hypothesis. Evaluating tree ensembles against deep learning (MLP, ResNet, FT-Transformer) across 45 medium-sized tabular datasets, they found that tree-based models dominate in nearly every setting. They attribute this to two properties of tabular data: individual features carry uninformative rotations in embedding space, which neural networks struggle to ignore; and real tabular datasets contain irregular target functions with sharp, non-smooth boundaries that tree splits represent naturally. Network traffic data has both properties: features like packet count and flag ratios are not compositional, and attack traffic often differs from benign traffic by hard thresholds on specific features rather than smooth gradients.

Isolation Forest and variants. Liu et al. [8] introduced Isolation Forest (IF), which isolates anomalies by randomly partitioning the feature space and measuring how few splits it takes to isolate a point. Points that isolate quickly are anomalous. The algorithm does not rely on density estimation, making it less sensitive to the curse of dimensionality than LOF. Hariri et al. [9] introduced Extended Isolation Forest (EIF), replacing axis-aligned splits with random hyperplanes at arbitrary orientations. This resolves a known geometric bias in standard IF where anomaly scores are artificially elevated near feature boundaries and at distribution centers, artifacts produced by axis-aligned cuts.

Hyperparameter optimization. Akiba et al. [4] introduced Optuna, a define-by-run framework for automatic hyperparameter search using Tree-structured Parzen Estimators (TPE). Xu and Liu [1] did not report hyperparameter tuning beyond fixed

configurations, introducing a comparison bias against models that are sensitive to hyperparameter choice.

Explainability. Lundberg and Lee [10] introduced SHAP (SHapley Additive exPlanations), a unified framework for feature attribution based on Shapley values from cooperative game theory. Tree SHAP [11] computes exact SHAP values for tree models in polynomial time. For IDS operators who must audit flagged alerts, interpretability is not optional: it determines whether a detection system can be deployed in regulated or high-stakes environments.

III. METHODOLOGY

A. Data Processing

We use CICIDS2017, which contains 2,830,743 flow records across 15 labeled categories. The class distribution is heavily skewed toward benign traffic (2,270,397 samples), with attack classes ranging from 231,073 (DoS Hulk) down to 11 (Heartbleed). We follow the holdout protocol of Xu and Liu [1] to permit direct comparison.

Three attack types are designated as zero-day unknowns and excluded from all training sets: DoS slowloris (5,796 samples), DoS Slowhttptest (5,499 samples), and Bot (1,966 samples). These were selected because their sample counts are large enough to produce stable test metrics while being small enough that their removal does not distort the training set composition.

The data is partitioned into three non-overlapping subsets:

Training set.

- Supervised models (Random Forest, LightGBM, XGBoost, CatBoost): 80% of benign samples plus 80% of each of the 11 non-holdout attack types. Labels are binary (benign = 0, malicious = 1).
- Unsupervised models (Isolation Forest, Extended Isolation Forest): 80% of benign samples only.

Overall Test Set. The remaining 20% of benign samples, 20% of the 11 known attack types, and all three zero-day attack types. This evaluates general detection performance across both seen and unseen traffic.

Unknown Attack Test Set. All samples from the three held-out zero-day attack types plus an equal number of benign samples drawn from the 20% benign test split. This isolates zero-day generalization performance.

All random sampling uses `random_state=42` for reproducibility. Preprocessing applies scikit-learn's `StandardScaler`, fit exclusively on the training set and applied without refitting to both test sets, to prevent data leakage. Columns containing infinite or NaN values (produced by CICFlowMeter in edge-case flows) are replaced with the column median computed on the training set.

B. Implementation Environment

All experiments were conducted on a single local workstation:

- **OS:** Windows 11 Home (Build 22H2.8246)
- **CPU:** AMD Ryzen 7 8845HS @ 3.80 GHz (8 cores / 16 threads)

- **GPU:** NVIDIA GeForce RTX 4060 Laptop (8 GB VRAM)
- **RAM:** 16 GiB
- **Python:** 3.11
- **scikit-learn:** 1.8.0
- **xgboost:** 3.2.0
- **catboost:** 1.2.10
- **lightgbm:** 4.6.0
- **optuna:** 4.8.0
- **pyod:** 2.0.2 (Extended IF)
- **shap:** 0.51.0
- **pandas:** 3.0.2, **numpy:** 2.4.4

C. Classifiers

1) *Random Forest*: Random Forest [12] constructs an ensemble of T decision trees, each trained on a bootstrap sample of the training data with a random feature subset of size $\sqrt{d}$ at each split. The ensemble prediction aggregates individual tree outputs:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}) \quad (1)$$

where $h_t(\mathbf{x})$ is the prediction of the t -th tree. Random feature subsampling decorrelates the individual trees, reducing ensemble variance without increasing bias. Final Optuna-tuned configuration: `n_estimators=150`, `max_depth=15`, `n_jobs=-1` (see Appendix A for all configurations).

2) *XGBoost*: XGBoost [5] minimizes a regularized objective over K additive trees:

$$\mathcal{L}(\Theta) = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (2)$$

where $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2$ penalizes the number of leaves T and the magnitude of leaf weights $\mathbf{w}$. The gain from a candidate split is:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (3)$$

where G_L, G_R are the sums of first-order gradients and H_L, H_R are the sums of second-order gradients in the left and right child nodes. A split is executed only when the gain is positive. Final Optuna configuration: `n_estimators=295`, `max_depth=5`, `learning_rate=0.1964`, `subsample=0.9189`, `tree_method='hist'`.

3) *LightGBM*: LightGBM [6] reduces training cost on large datasets through two mechanisms. Gradient-based One-Side Sampling (GOSS) retains all samples with large gradients (fraction a) and randomly draws a fraction b of small-gradient samples, amplifying them by $(1-a)/b$ to correct for sampling bias:

$$\tilde{V}_j(d) = \frac{1}{n} \left(\sum_{x_i \in A_l} g_i + \frac{1-a}{b} \sum_{x_i \in B_l} g_i \right)^2 + \dots \quad (4)$$

Exclusive Feature Bundling (EFB) merges mutually exclusive sparse features into single dense features, reducing effective

dimensionality without information loss. The leaf-wise growth strategy expands the leaf with the largest loss reduction at each step, producing deeper, more expressive trees for the same number of leaves. Final Optuna configuration: `n_estimators=269`, `max_depth=5`, `learning_rate=0.1846`, `num_leaves=144`.

4) *CatBoost*: CatBoost [7] addresses the prediction shift problem in standard gradient boosting, where the model used to compute residuals for sample x_i was trained on a dataset that includes x_i itself, a form of target leakage. Ordered boosting eliminates this by maintaining a separate model for each random permutation σ and computing the residual for sample $\sigma(j)$ using only the $j-1$ preceding samples in that permutation:

$$r(\sigma(j), \sigma) = F_{\sigma, j-1}^K(x_{\sigma(j)}) - y_{\sigma(j)} \quad (5)$$

where $F_{\sigma, j-1}^K$ denotes the ensemble built from the first $j-1$ samples. CatBoost further uses oblivious trees (symmetric decision trees where the same split condition applies across all nodes at a given depth), which accelerates inference and provides implicit regularization. Final Optuna configuration: `iterations=125`, `depth=8`, `learning_rate=0.1986`.

5) *Isolation Forest*: Isolation Forest [8] detects anomalies by measuring isolation depth: it builds an ensemble of t isolation trees, each constructed by recursively choosing a random feature q and a random split value p from $[\min(q), \max(q)]$ until every sample is isolated. Anomalies require fewer splits because they are sparse relative to surrounding data. The anomaly score for sample $\mathbf{x}$ is:

$$s(\mathbf{x}, n) = 2^{-\frac{\mathbb{E}[h(\mathbf{x})]}{c(n)}} \quad (6)$$

where $h(\mathbf{x})$ is the path length to isolation and $c(n)$ normalizes by the expected path length of an unsuccessful binary search:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) = \ln(i) + 0.5772 \dots \quad (7)$$

Scores near 1 indicate anomalies; scores near 0.5 indicate normal points. Final configuration: `n_estimators=200`, `max_samples='auto'`, `contamination=0.05`, `decision_threshold=-0.2120`.

6) *Extended Isolation Forest*: Standard IF has a geometric bias: axis-aligned splits produce anomaly score artifacts near axis boundaries and at distribution centers regardless of actual data density [9]. Extended Isolation Forest (EIF) corrects this by replacing axis-aligned splits with random hyperplane cuts. At each branching node, a random normal vector $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and a random intercept p drawn from the projected data range determine the split:

$$\text{split condition: } \mathbf{x} \cdot \mathbf{n} + p \leq 0 \quad (8)$$

Because $\mathbf{n}$ is unconstrained to a single feature axis, EIF produces cuts at arbitrary orientations, eliminating the score artifacts of standard IF. We used the `pyod` implementation. Final configuration: `n_estimators=100`,

max_samples=256, contamination=0.05, decision threshold = -0.1815.

D. Model Selection Justification

The model selection targets two gaps in the Xu and Liu benchmark. The supervised gap: the benchmark compared two deep learning models (MLP, CNN) against two unsupervised models (OCSVM, LOF) but included no supervised tree ensemble. Given Grinsztajn et al.’s [3] finding that trees outperform deep learning on tabular data across 45 datasets, this omission biases the benchmark toward deep learning. The unsupervised gap: OCSVM uses a single RBF kernel boundary that may not fit the heterogeneous structure of zero-day traffic, while LOF suffers from the curse of dimensionality in 80-dimensional feature spaces. Isolation Forest and Extended Isolation Forest were designed for exactly these high-dimensional, heterogeneous settings.

E. Evaluation Metrics

We report the same binary classification metrics as Xu and Liu [1] for direct comparability. Class 0 denotes benign traffic; class 1 denotes malicious traffic.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (9)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (10)$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (11)$$

In the IDS setting, recall carries particular weight because a missed attack (false negative) typically has higher operational cost than a benign flow flagged for inspection (false positive). F1-score serves as the primary ranking metric.

IV. RESULTS

A. Known Attacks: Overall Test Set

Table I presents performance on the Overall Test Set, which evaluates general detection across both seen and unseen traffic. This matches the evaluation protocol of Xu and Liu [1].

TABLE I: Performance on the Overall Test Set

Model	Acc.	Prec.	Recall	F1
<i>Xu & Liu Baselines</i>				
MLP	0.9775	0.9894	0.9036	0.9446
CNN	0.9650	0.9316	0.9010	0.9160
LOF	0.8046	0.9106	0.8341	0.8706
OCSVM	0.8356	0.6525	0.4784	0.5520
<i>Tree-Based Models (Ours)</i>				
Random Forest	0.9784	0.9970	0.9010	0.9465
LightGBM	0.9841	0.9977	0.9271	0.9610
XGBoost	0.9850	0.9977	0.9318	0.9635
CatBoost	0.9853	0.9975	0.9331	0.9643

All four tree models exceed the MLP baseline on accuracy and F1-score. The margin is largest for CatBoost (F1 = 0.9643 vs. 0.9446, +0.020) and smallest for Random Forest (+0.002). LightGBM, XGBoost, and CatBoost each achieve recall above

0.927, while MLP achieves 0.904. Precision is uniformly high across all tree models (>0.997), exceeding MLP’s 0.989. The gradient-boosted variants show the clearest improvement, while Random Forest’s gain is marginal, consistent with Grinsztajn et al.’s [3] finding that boosted trees outperform bagging ensembles on structured tabular data.

Among the boosted variants, CatBoost’s ordered boosting provides a measurable advantage over XGBoost and LightGBM. The class imbalance in CICIDS2017 (benign samples outnumber attacks roughly 9:1) likely contributes: CatBoost’s target leakage prevention reduces overfitting to majority-class patterns that can affect standard gradient boosting under imbalanced conditions.

Fig. 1 visualizes the F1-score comparison across all supervised models on the Overall Test Set.

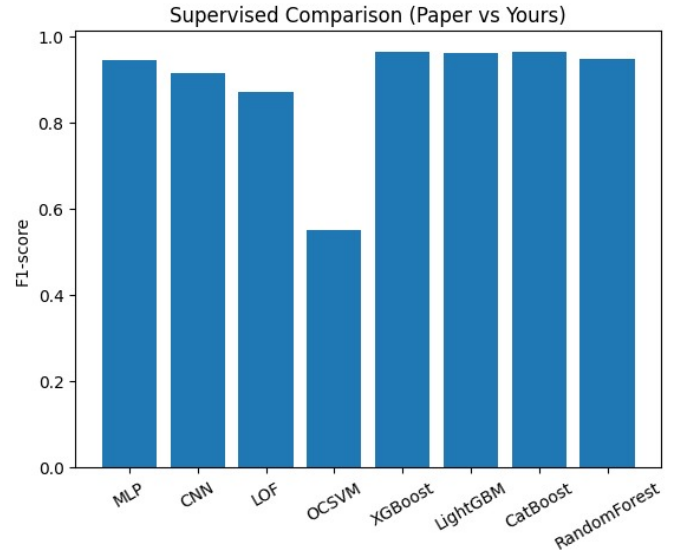


Fig. 1: F1-score comparison of supervised models on the Overall Test Set. All four tree-based models exceed both MLP and CNN baselines. CatBoost achieves the highest F1 of 0.9643.

Confusion Matrix Analysis: Fig. 2 shows confusion matrices for all four supervised tree models on the 576,758-sample Overall Test Set.

The confusion matrices reveal performance differences that aggregate metrics partially obscure. CatBoost (Fig. 2d) achieves near-perfect separation across 576,758 test samples: only 280 false positives and 8,170 false negatives. This tight error budget explains its state-of-the-art F1 of 0.9643, as both error types are simultaneously minimized rather than one being traded off against the other. The 280 false positives represent benign flows that fall close enough to the attack boundary to be misclassified; the 8,170 false negatives are predominantly from low-volume or ambiguous attack categories that do not deviate far enough from normal traffic in the feature space CatBoost learned.

Random Forest (Fig. 2a) presents a different profile. Its false positive count is comparably low (a consequence of its high

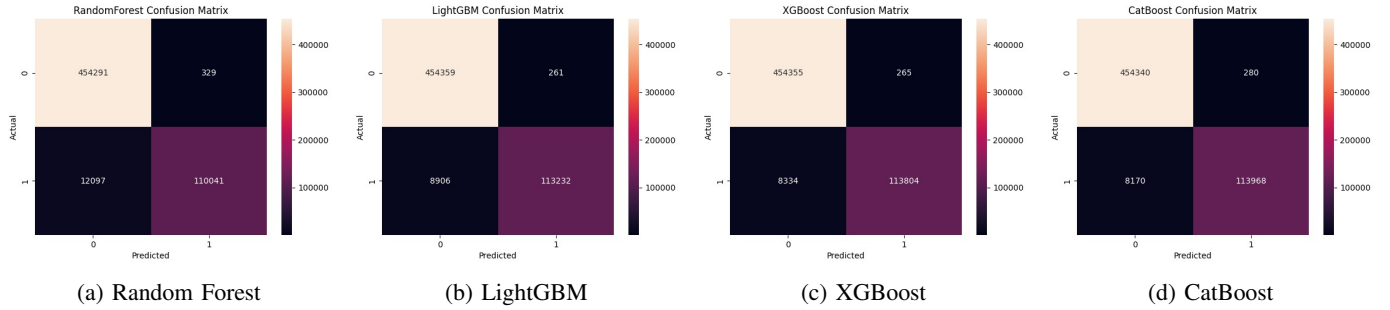


Fig. 2: Confusion matrices on the Overall Test Set (576,758 samples). (a)–(d) correspond to Random Forest, LightGBM, XGBoost, and CatBoost, respectively.

precision of 0.9970), but it accumulates 12,097 false negatives, nearly 50% more than CatBoost. This gap traces directly to the difference between bagging and ordered boosting: Random Forest’s independently trained trees vote without any mechanism to correct residuals on hard cases, while CatBoost’s sequential, bias-corrected boosting iteratively reduces the false negative rate on samples that earlier trees misclassify. LightGBM and XGBoost sit between these extremes, with false negative counts that decrease as the boosting framework becomes more expressive.

B. Zero-Day Attacks: Unknown Attack Test Set

Table II presents performance on the Unknown Attack Test Set, which contains only the three held-out zero-day attack types and an equal number of benign samples.

TABLE II: Performance on the Unknown Attack Test Set

Model	Acc.	Prec.	Recall	F1
<i>Xu & Liu Baselines</i>				
MLP	0.5863	0.9860	0.1750	0.2973
CNN	0.5882	0.9110	0.1954	0.3218
LOF	0.6087	0.5746	0.8370	0.6814
OCSVM	0.7919	0.9072	0.6503	0.7575
<i>Tree-Based Models (Ours)</i>				
Unsup. XGBoost	0.6712	0.9717	0.3526	0.5175
Isolation Forest	0.7121	0.6349	0.9985	0.7762
Ext. IF (EIF)	0.7468	0.6865	0.9085	0.7821

Supervised model collapse. MLP and CNN both fail on zero-day attacks despite near-perfect performance on the overall test set. MLP’s recall drops from 0.9036 to 0.1750: it detects fewer than one in five zero-day malicious flows. CNN shows the same pattern (recall = 0.1954). Both models learned discriminative boundaries between benign and labeled attack traffic; zero-day traffic falls in regions the boundary was never trained to classify as malicious, so the models default to benign predictions. The high precision scores (MLP: 0.986; CNN: 0.911) are artifacts of this behavior. Among the small fraction each model does flag as malicious, it is usually right, but it flags almost nothing.

Unsupervised baselines. LOF achieves high recall (0.8370) but low precision (0.5746), producing $F1 = 0.6814$. Its density-based scoring in 80-dimensional feature space is noisy and

generates numerous false positives. OCSVM ($F1 = 0.7575$) offers a better precision-recall balance (precision = 0.9072, recall = 0.6503), consistent with Xu and Liu’s finding that its global RBF boundary generalizes better than LOF’s local density estimates.

Tree-based anomaly detection. Isolation Forest achieves near-perfect recall of 0.9985, detecting virtually all zero-day flows, but at the cost of lower precision (0.6349), giving $F1 = 0.7762$. EIF improves on this substantially: precision rises to 0.6865 and recall remains high at 0.9085, producing $F1 = 0.7821$, which exceeds OCSVM’s $F1$ by 0.025. The precision gain from IF to EIF (+5.2 points) reflects the correction of geometric bias. Standard IF’s axis-aligned splits assign inflated anomaly scores to benign samples near feature-space boundaries, a systematic false positive source that EIF’s oblique hyperplanes eliminate by scoring those samples against a fairer decision surface.

DoS slowloris and DoS Slowhttptest generate flows with low-rate, persistent connection patterns: moderate packet counts with unusually long inter-arrival times. Bot traffic shows coordinated, rhythmic connection patterns. Neither type differs from benign traffic along individual feature axes in a way axis-aligned cuts can isolate cleanly; the anomaly lies in feature combinations. Oblique cuts in EIF form decision surfaces that capture these multivariate deviations directly, while IF’s axis-parallel cuts require deeper trees to approximate the same boundaries.

Fig. 3 visualizes the $F1$ -score comparison across all unsupervised models on the Unknown Attack Test Set.

Confusion Matrix Analysis: Fig. 4 shows confusion matrices for the three unsupervised tree models on the Unknown Attack Test Set.

Standard Isolation Forest (Fig. 4b) is extraordinarily sensitive: it produces only 20 false negatives, missing fewer than 0.2% of zero-day attack flows. This sensitivity comes at a cost of 7,615 false positives: benign flows that score below the -0.2120 decision threshold because their feature combinations fall in regions that axis-aligned cuts isolated quickly. In a live deployment where an analyst reviews each alert, 7,615 false positives per test cycle would saturate the triage queue.

Extended Isolation Forest (Fig. 4c) resolves this directly. False positives drop from 7,615 to 5,501 (a 27.8% reduction)

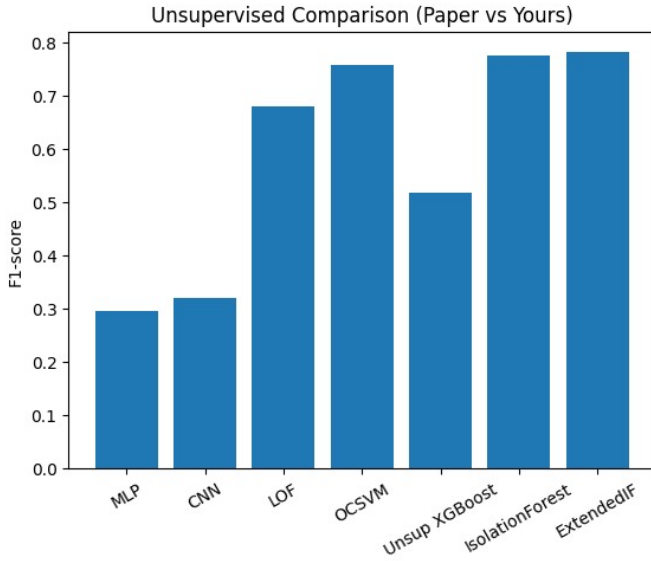


Fig. 3: F1-score comparison of unsupervised models on the Unknown Attack Test Set. EIF (0.7821) surpasses OCSVM (0.7575) while maintaining high recall, confirming that oblique hyperplane splits generalize better to zero-day attack patterns than a single RBF kernel boundary.

at the cost of raising false negatives from 20 to 1,213. The net effect is a tighter, operationally useful boundary: EIF still catches 90.8% of zero-day attacks while generating roughly 2,100 fewer false alarms per cycle than IF. This improvement traces to the oblique hyperplane mechanism: random normal vectors in $\mathbb{R}^{80}$ can cut across the diagonal structure of benign traffic clusters, placing many borderline benign flows correctly inside the “normal” region that axis-aligned splits could not contain.

The Unsupervised XGBoost model (Fig. 4a), trained via pseudo-anomaly generation with a decision threshold of 0.0000, achieves high precision (0.9717) but collapses in recall (0.3526) on the three zero-day types. As reported in Table II and reflected in the bar chart of Fig. 3, its F1 of 0.5175 falls well below both IF and EIF, placing it closer to the catastrophic collapse of MLP and CNN than to the genuine anomaly detectors. This confirms that pseudo-anomaly generation, which synthesizes artificial outliers to give the unsupervised model a notional negative class during training, does not produce a decision boundary that transfers to genuinely novel attack patterns. The synthetic anomalies approximate the complement of benign traffic as observed in the training set; zero-day attacks that differ from training traffic in ways not captured by that synthetic distribution pass through the boundary undetected.

C. Feature Explainability via SHAP

A concern absent from the Xu and Liu benchmark is whether model decisions can be audited. Security analysts need to understand why a specific flow was flagged: to triage alerts, write detection rules, and satisfy compliance

requirements. Black-box deep learning models do not provide this directly.

We computed Tree SHAP [11] attributions for all five tree models on 5,000 randomly sampled test predictions. Fig. 5 shows mean absolute SHAP values for the three supervised models, and Fig. 6 shows the same for the two unsupervised anomaly detectors.

Supervised models: signature-driven detection. The SHAP plots for CatBoost, LightGBM, and XGBoost (Fig. 5) show a consistent pattern: Destination Port and Init_Win_bytes_forward (the initial TCP window size in the forward direction) receive the highest mean absolute SHAP values across all three models.

The dominance of Destination Port is analytically significant. The supervised training set includes attacks such as FTP-Patator (port 21), SSH-Patator (port 22), and Web Attacks (port 80 or 443). When a tree splits on Destination Port = 21 to isolate FTP-Patator traffic, it is not learning a behavioral anomaly: it is memorizing an attack identity encoded as a port number. This is functionally equivalent to a signature rule, implemented through a tree split rather than a string match.

Init_Win_bytes_forward reinforces this interpretation. The initial TCP window size is highly attack-tool-specific: exploit frameworks and attack scripts often use non-standard window sizes that differ from OS defaults (Windows typically advertises 65,535 bytes; Linux 29,200 bytes). A split on this feature at training time captures a behavioral fingerprint of specific known attack tools, not a general property of malicious traffic.

This explains why supervised tree models collapse on zero-day attacks (recall dropping to $\sim 20\%$ for MLP, with similar patterns in trees when evaluated on truly unseen types): DoS slowloris, DoS Slowhttptest, and Bot traffic operate on standard ports (80, 443) and use standard TCP window advertisements. The two features that dominate supervised detection carry zero discriminating signal for these zero-day types, so the decision boundary learned during training provides no useful guidance.

Unsupervised models: behavior-driven detection. The SHAP plots for Isolation Forest and EIF (Fig. 6) show a completely different structure. Bwd Packet Length Max (the maximum backward packet length in a flow) and Flow IAT Max (the maximum inter-arrival time between packets in the flow) receive the highest attributions. Neither feature encodes attack identity; both measure physical properties of network behavior.

Bwd Packet Length Max captures anomalous response packet sizes. Normal browsing traffic generates variable but bounded backward packet lengths, driven by page content and server response sizes. DoS slowloris attacks, which hold connections open by sending partial HTTP requests, generate very small or absent backward responses, with maximum backward packet lengths that fall far outside the normal distribution learned from training traffic. Bot traffic, which communicates in structured command-and-control

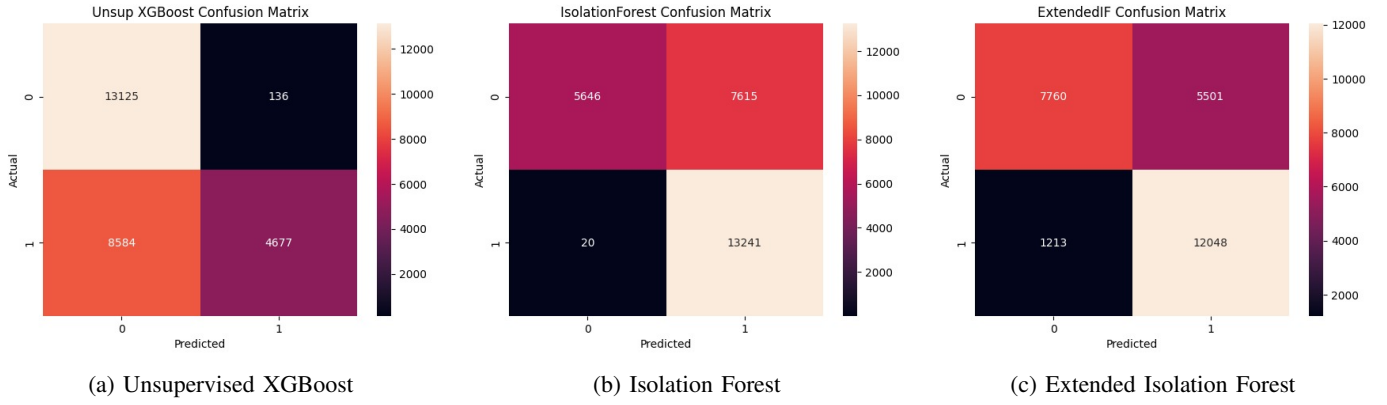


Fig. 4: Confusion matrices on the Unknown Attack Test Set. (a)–(c) correspond to Unsupervised XGBoost, Isolation Forest, and Extended Isolation Forest.

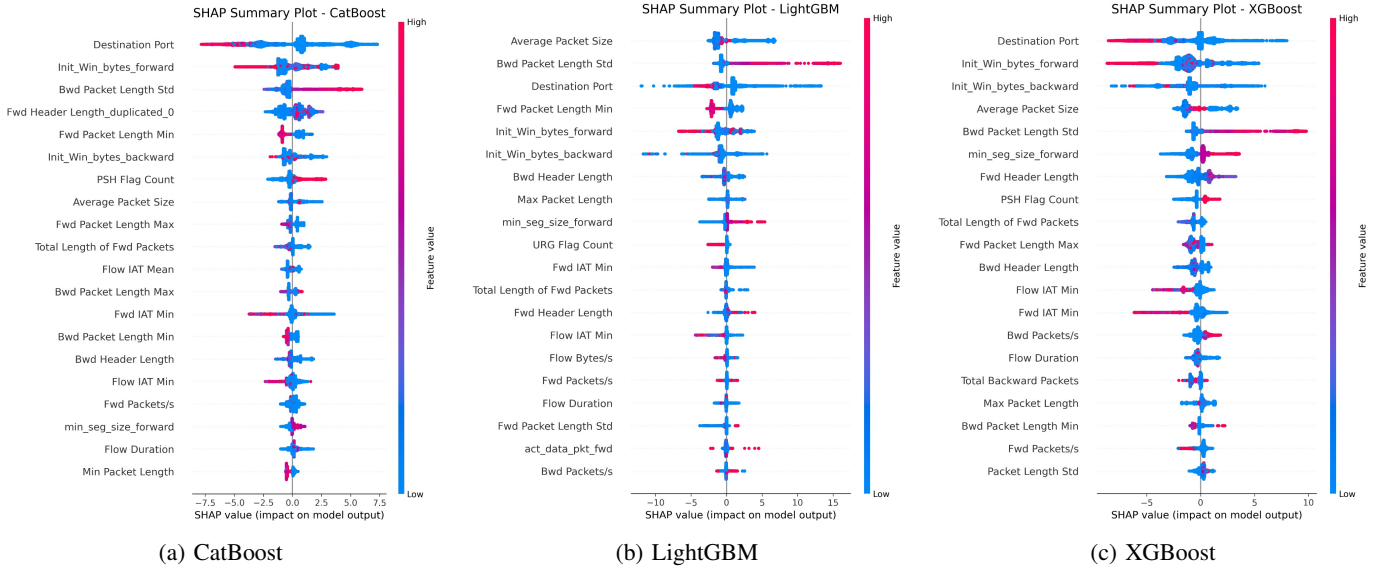


Fig. 5: SHAP mean absolute feature importance for supervised tree models. All three models assign the highest attribution to Destination Port and Init_Win_bytes_forward, reflecting signature-based detection logic.

bursts, generates backward packets of atypical uniform sizes inconsistent with legitimate server responses.

Flow IAT Max captures timing irregularities. The maximum inter-arrival time in a flow is a direct measure of how long a connection remained open without activity. DoS slowloris and DoS Slowhttptest attacks maintain connections at abnormally long intervals, sending just enough traffic to prevent connection timeout while never completing the HTTP request. This feature has a characteristic distribution in normal traffic (bounded by server timeout policies and human interaction rhythms) and both DoS slow-rate attacks push it to extreme values. Bot heartbeat connections produce similarly irregular inter-arrival patterns.

The contrast between the two paradigms is fundamental: supervised models learn what specific attacks look like (which ports they use, which tool fingerprints they carry), while unsupervised models learn what normal traffic physically looks like

and flag deviations from that pattern. This structural difference fully explains the zero-day generalization gap. When DoS slowloris and Bot traffic arrive at inference time, supervised models see traffic on familiar ports with familiar window sizes and classify it as benign. Unsupervised models see flows with backward packet lengths and inter-arrival times that were never observed in the benign training distribution and correctly flag them as anomalous.

These SHAP attributions align with established domain knowledge in the IDS literature [2], [19]. Flow timing and packet size asymmetry are standard indicators in manual signature construction for slow-rate DoS and botnet categories. The fact that SHAP recovers them for the unsupervised models, without domain supervision, confirms that IF and EIF have learned the operationally relevant behavioral properties of normal traffic. An analyst auditing an EIF alert that cites high Bwd Packet Length Max deviation has

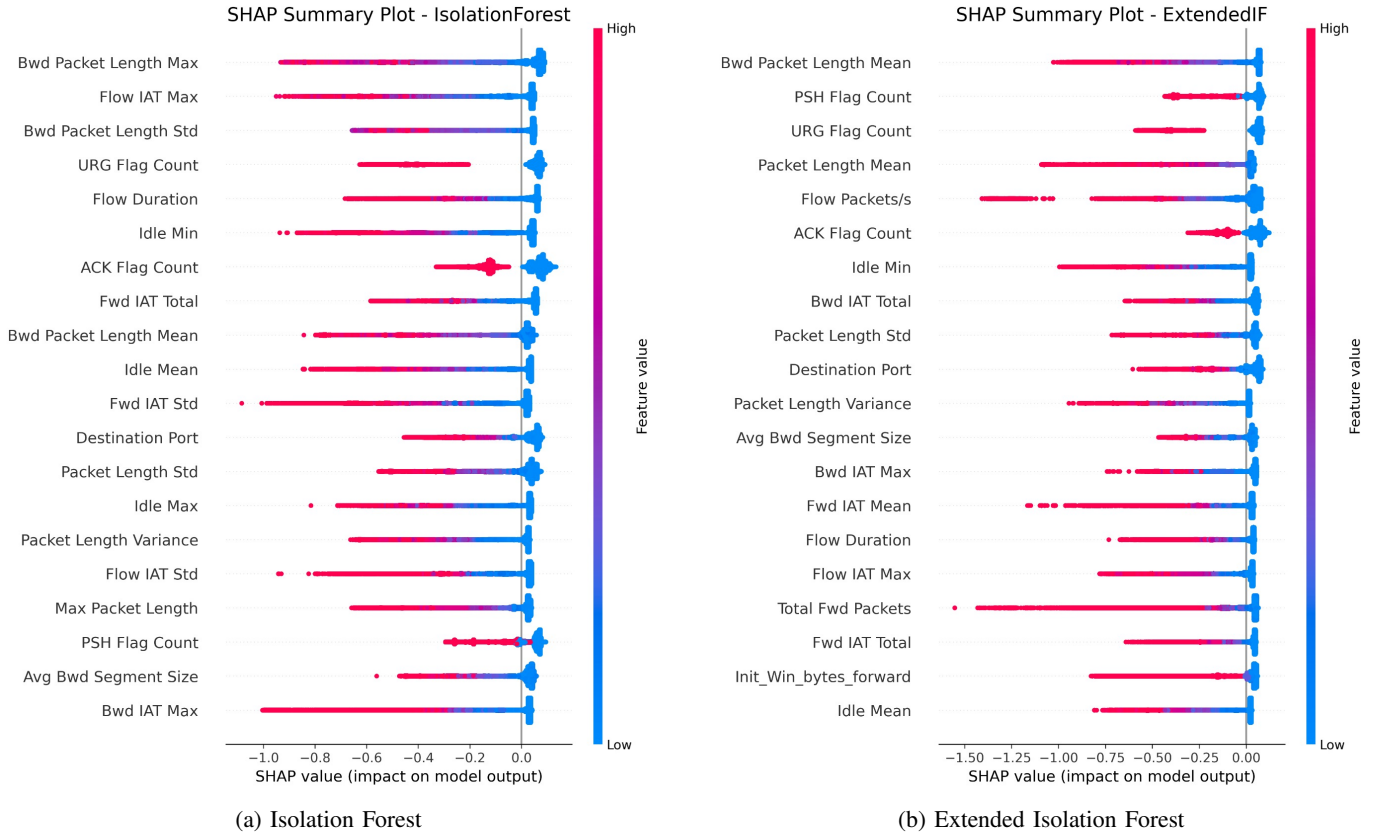


Fig. 6: SHAP mean absolute feature importance for unsupervised tree models. Both models assign dominant attribution to Bwd Packet Length Max and Flow IAT Max, reflecting anomaly-based detection logic grounded in physical traffic behavior.

a clear, actionable explanation: an unusual response pattern from this connection. An analyst auditing an MLP alert has no comparable feature-level explanation without additional tooling.

V. DISCUSSION AND CONCLUSION

The results confirm the double-victory hypothesis across both evaluation scenarios. On known attacks, CatBoost ($F1 = 0.9643$) exceeds MLP ($F1 = 0.9446$) by two percentage points. On zero-day attacks, EIF ($F1 = 0.7821$) exceeds OCSVM ($F1 = 0.7575$) by 2.5 points. Both results follow from Grinsztajn et al. [3]: CICIDS2017’s features are tabular, heterogeneous, and non-compositional, the regime where trees outperform deep learning.

Why CatBoost beats MLP. Trees do not require the implicit feature interaction structure that neural networks exploit through learned embeddings. Each split operates on a single feature, matching the semantics of network flow statistics directly. CatBoost’s ordered boosting adds a second advantage: it prevents the systematic bias from target leakage that affects gradient boosting on imbalanced datasets, and CICIDS2017’s class imbalance (benign samples are roughly 80% of the full dataset) makes this correction consequential.

Why EIF beats OCSVM. OCSVM builds a single global decision boundary in RBF kernel space. For zero-day attacks

that cluster densely in specific feature-space regions, this global boundary either encloses those regions as part of “normal” space or excludes them at the cost of false positives on surrounding benign traffic. EIF’s ensemble of oblique cuts handles density-varying feature spaces more adaptively: each isolation tree constructs an independent view of the feature space, and the ensemble average smooths individual tree errors. The random hyperplane orientations allow EIF to form compact boundaries around normal traffic clusters without being constrained to axis-aligned shapes.

The SHAP paradigm insight. The SHAP analysis adds a layer of explanation that goes beyond metrics. Supervised models are effectively learning enriched signatures: port numbers and TCP window sizes are encoded attack identities compressed into tree splits. This produces excellent recall on known attacks but zero generalization to attacks that use standard ports. Unsupervised models learn behavioral norms: packet size distributions and timing patterns that are stable properties of legitimate traffic. This produces lower recall on known attacks (which may not violate behavioral norms) but genuine generalization to zero-day traffic that does.

Deployment recommendation. For organizations deploying IDS on tabular network flow data, we recommend a two-stage architecture. CatBoost trained on labeled benign and known-attack traffic serves as the primary fast filter. It

handles the vast majority of traffic (both benign and familiar attack types) with precision exceeding 0.997 and recall above 0.933. Flows that CatBoost classifies as benign with low confidence are passed to EIF, which was trained exclusively on benign traffic and flags behavioral outliers regardless of attack category. This ensemble covers both the known-attack and zero-day scenarios while providing SHAP-based explanations for every flagged alert.

Limitations. CICIDS2017 was generated in 2017 under controlled laboratory conditions and does not capture contemporary attack traffic distributions. The three zero-day categories here were chosen to match the Xu and Liu protocol; a different holdout selection would change absolute scores, though we expect the relative ranking to hold. The SHAP analysis is correlational: it shows which features the models rely on but does not establish causal links to attack mechanisms. Future work should validate these findings on more recent datasets (CIC-IDS2018, CIC-Bell-DNS-EXF-2021) and test whether the double-victory pattern holds under distribution shift.

Conclusion. The deep-learning-centric view in current IDS research does not hold once tree-based methods are included. CatBoost sets a new state-of-the-art on CICIDS2017 for known-attack detection. Extended Isolation Forest does the same for zero-day detection. SHAP analysis reveals that these are not interchangeable models: supervised trees encode attack signatures and excel on known threats; unsupervised trees encode traffic norms and excel on novel ones. A hybrid IDS that deploys both in sequence leverages the complementary strengths of each paradigm.

REFERENCES

- [1] Z. Xu and Y. Liu, "Robust anomaly detection in network traffic: Evaluating machine learning models on CICIDS2017," arXiv preprint arXiv:2506.19877, 2025.
- [2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [3] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why tree-based models still outperform deep learning on tabular data," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 507–520.
- [4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 6638–6648.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [9] S. Harii, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1479–1489, 2019.
- [10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

- [11] S. M. Lundberg, G. Erion, and S.-I. Lee, "Consistent individualized feature attribution for tree ensembles," arXiv preprint arXiv:1802.03888, 2018.
- [12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] T. Elmasri, N. Samir, M. Mashaly, and Y. Atef, "Evaluation of CICIDS2017 with qualitative comparison of machine learning algorithm," in *Proc. 2020 IEEE Cloud Summit*, 2020, pp. 46–51.
- [14] S. Hizal, Ü. Çavuşoğlu, and D. Akgün, "A new deep learning based intrusion detection system for cloud security," in *Proc. 3rd Int. Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)*, 2021, pp. 1–4.
- [15] Y. Wang and X. Yang, "Research on enhancing cloud computing network security using artificial intelligence algorithms," arXiv preprint arXiv:2502.17801, 2025.
- [16] C. Wang, C. Nie, and Y. Liu, "Evaluating supervised learning models for fraud detection: A comparative study of classical and deep architectures on imbalanced transaction data," arXiv preprint arXiv:2505.22521, 2025.
- [17] R. Kale, Z. Lu, K. W. Fok, and V. L. L. Thing, "A hybrid deep learning anomaly detection framework for intrusion detection," in *Proc. 2022 IEEE 8th Int. Conf. Big Data Security on Cloud (BigDataSecurity)*, 2022, pp. 137–142.
- [18] S. Tossou, M. Qorib, and T. Kacem, "Anomaly based intrusion detection system: A deep learning approach," in *Proc. 2023 Int. Symposium on Networks, Computers and Communications (ISNCC)*, 2023, pp. 1–6.
- [19] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [20] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, p. Article 15, 2009.
- [21] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

APPENDIX

Table III lists all final hyperparameter configurations selected by Optuna. Supervised models used 50 Optuna trials each, maximizing validation F1-score on a 20% stratified held-out split of the training data. Random Forest used 30 trials. Unsupervised models tuned contamination and decision threshold over the same held-out benign validation split.

TABLE III: Final Optuna-Tuned Hyperparameter Configurations

Model	Final Parameters
Random Forest	n_estimators=150 max_depth=15 n_jobs=-1
XGBoost	n_estimators=295 max_depth=5 learning_rate=0.1964 subsample=0.9189 tree_method='hist'
LightGBM	n_estimators=269 max_depth=5 learning_rate=0.1846 num_leaves=144
CatBoost	iterations=125 depth=8 learning_rate=0.1986
Unsup. XG-Boost	n_estimators=200 max_depth=6 threshold=0.0000
Isolation Forest	n_estimators=200 max_samples='auto' contamination=0.05 decision_threshold=-0.2120
Ext. IF (EIF)	n_estimators=100 max_samples=256 contamination=0.05 decision_threshold=-0.1815